

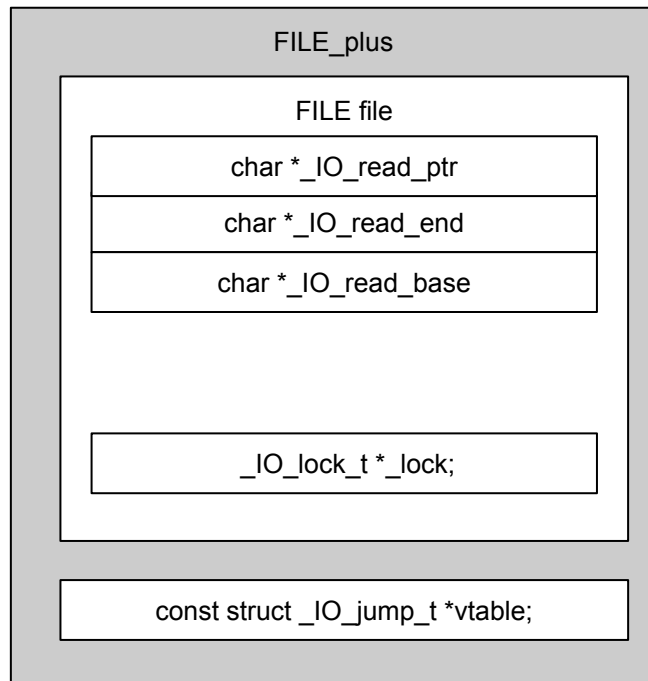
pwn.college: File Struct

FILE_plus and the vtable

Robert Wasinger
Arizona State University

_IO_FILE_plus

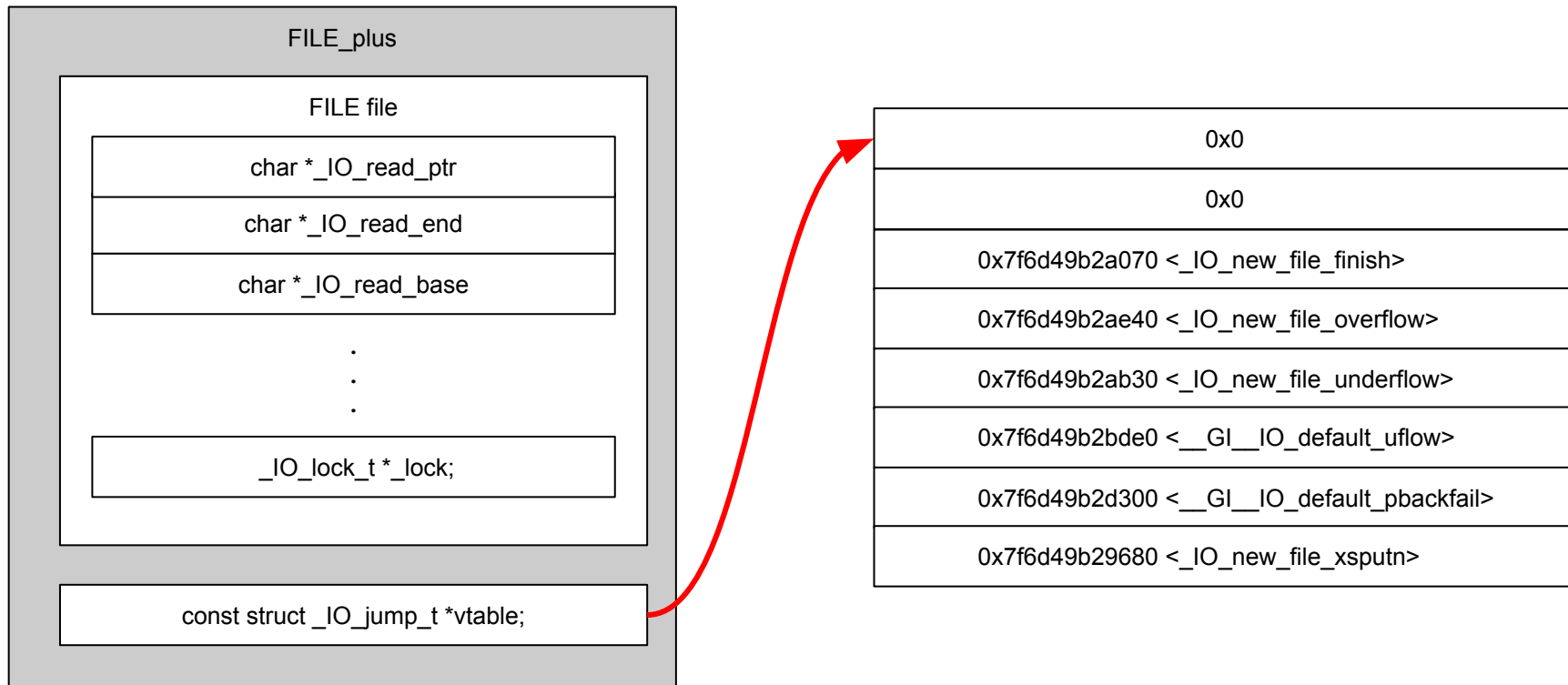
```
struct _IO_FILE_plus  
{  
    FILE file;  
  
    const struct _IO_jump_t *vtable;  
};
```



Virtual function table

- A table full of function pointers
- Commonly found in C++ binaries
- Allows for dynamic function resolution at runtime

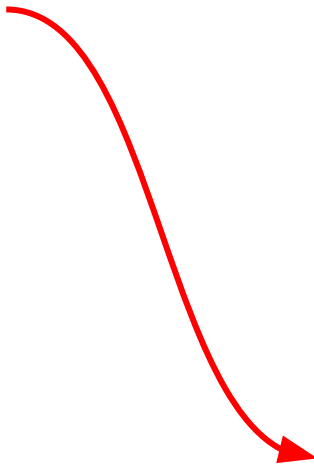
FILE_plus vtable pointer



FILE_plus vtable pointer

Inside fwrite...

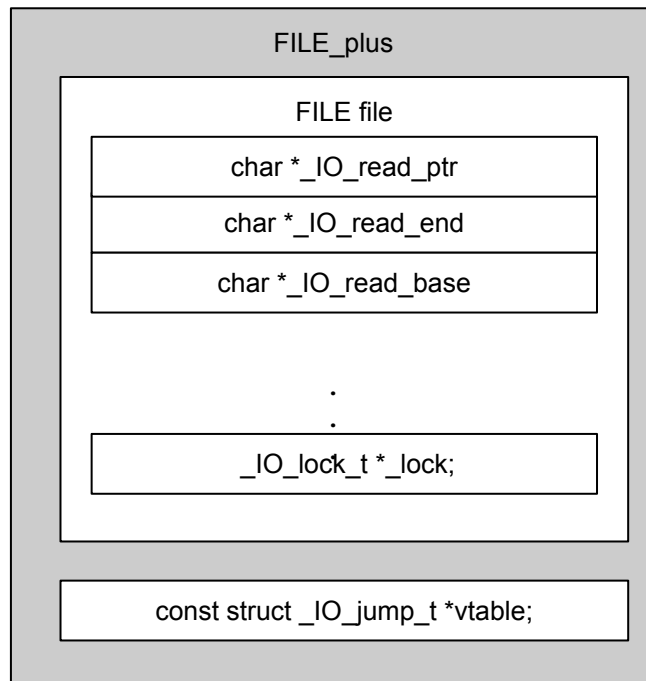
```
call QWORD PTR [r15+0x38]
```



0x0
0x0
0x7f6d49b2a070 <_IO_new_file_finish>
0x7f6d49b2ae40 <_IO_new_file_overflow>
0x7f6d49b2ab30 <_IO_new_file_underflow>
0x7f6d49b2bde0 <__GI__IO_default_uflow>
0x7f6d49b2d300 <__GI__IO_default_pbackfail>
0x7f6d49b29680 <_IO_new_file_xsputn>

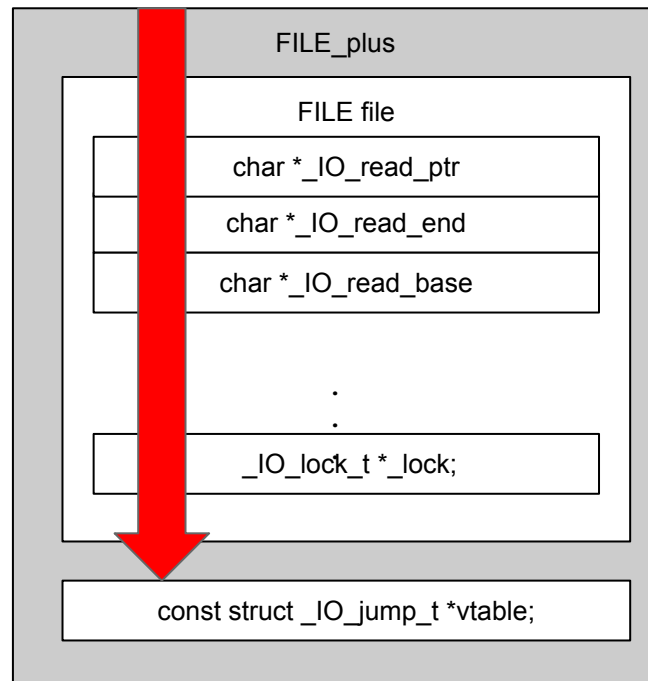
Exploiting the vtable pointer

- Create our own “exploit_vtable”
- Place desired execution address at correct offset
- Make `file.vtable` to point to `exploit_vtable`



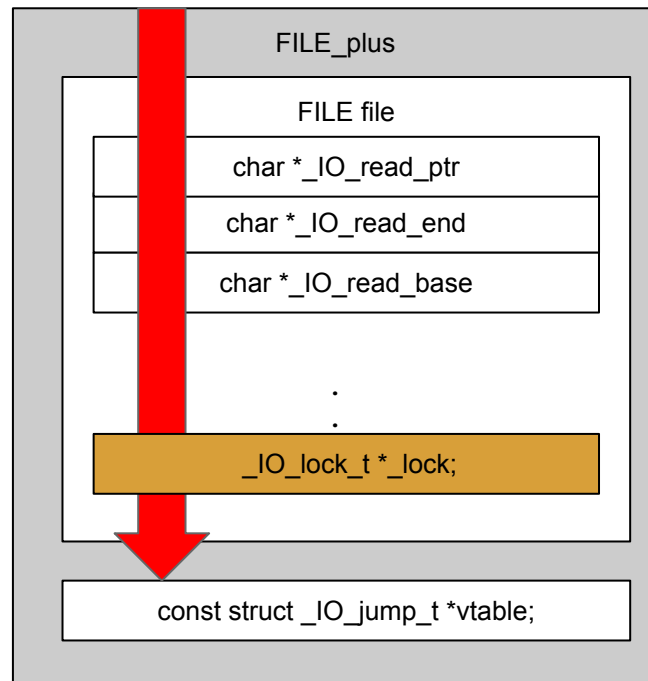
Exploiting the vtable pointer

- Create our own “exploit_vtable”
- Place desired execution address at correct offset
- Make `file.vtable` to point to `exploit_vtable`



Exploiting the vtable pointer

- Create an `exploit_vtable`
- Place desired execution address at correct offset
- Overwrite `file.vtable` to `exploit_vtable`

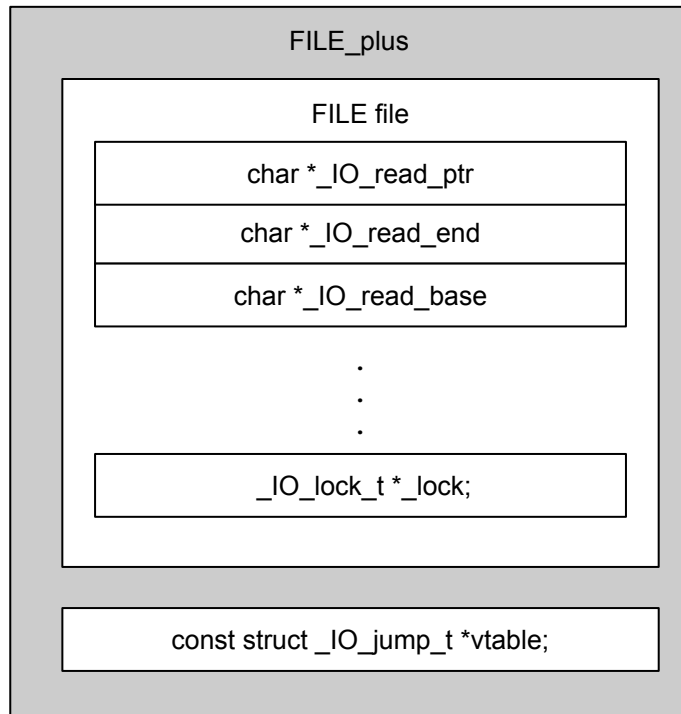


Exploiting the vtable pointer - `_lock`

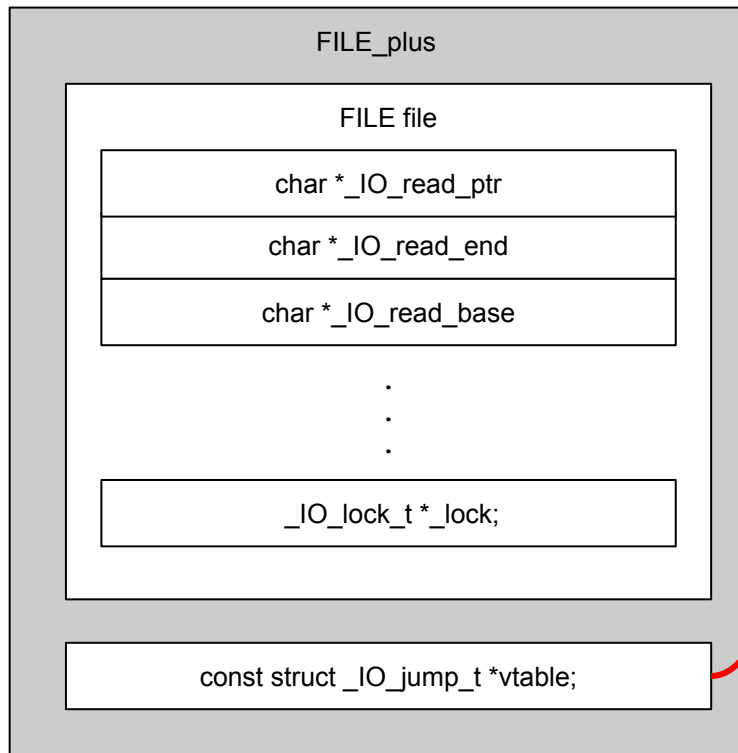
The `_lock` pointer is used for threaded file access

An exploit **MUST** set `_lock` to point to a

- writable location
- with value NULL



Exploiting the vtable pointer.. Almost

[illegible]

Exploiting the vtable pointer.. Almost

Inside fwrite..

```
call QWORD PTR [r15+0x38]
```

[illegible]

Exploiting vtable pointer.. Almost

No!

Modern libc performs a vtable pointer validation step

<https://elixir.bootlin.com/glibc/glibc-2.31/source/libio/libioP.h#L935>

Fully Exploiting the vtable pointer

- The vtable pointer must point to libc vtable area
 - Many IO related vtables next to each other
 - Many interesting candidates

Try and call `IO_wfile_overflow`

- Which calls `do_allocbuf..`
- Which uses another vtable (in `file->wide_data`) without verification!

wide_data

`wide_data` is yet another field that exists inside of modern FILE structs

- Created to handle wide character streams (unicode)
- Contains fields very similar to FILE
- Including a vtable pointer!

<https://elixir.bootlin.com/glibc/glibc-2.31/source/libio/libio.h#L121>

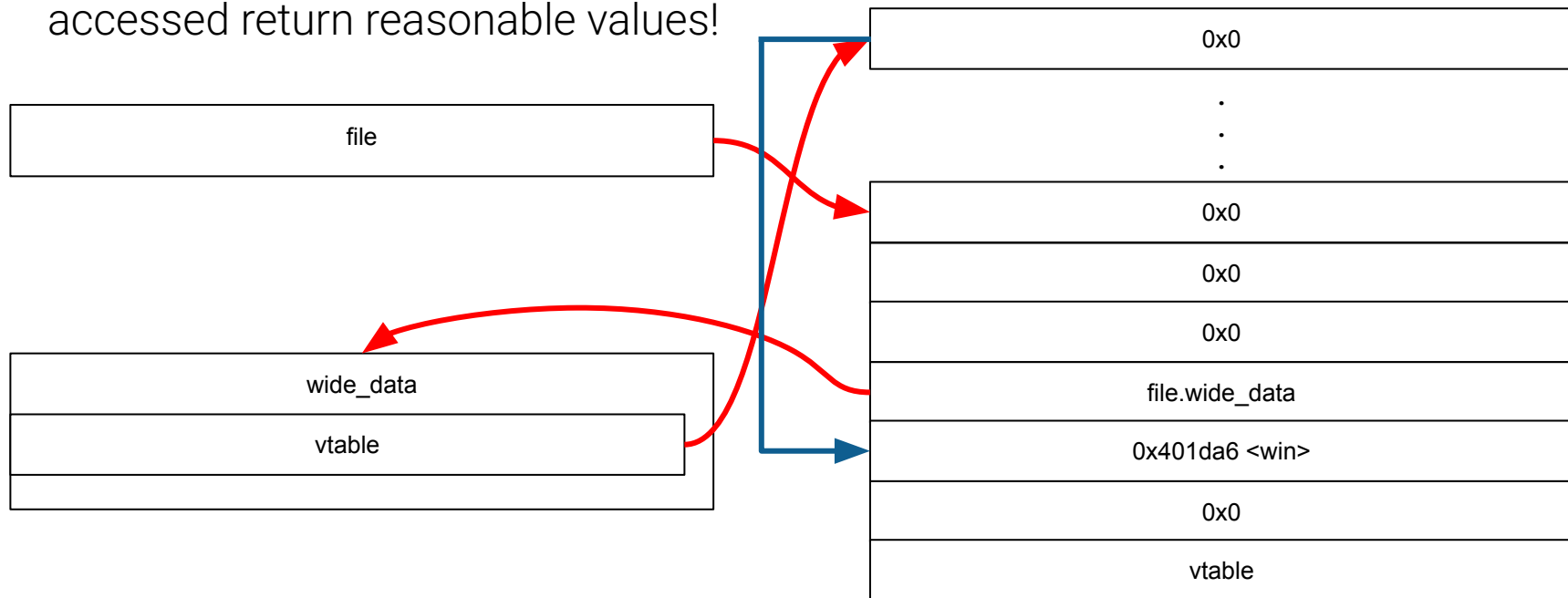
Fully Exploiting the vtable pointer

- Set `file.wide_data->vtable` to point to `exploit_vtable`
- Overwrite `file.vtable` such that `IO_wfile_overflow` gets called
- Which calls `do_allocbuf..`
- Which calls into the `wide_data` vtable with no check!

Let's explore this idea in gdb.

Overlapping structs

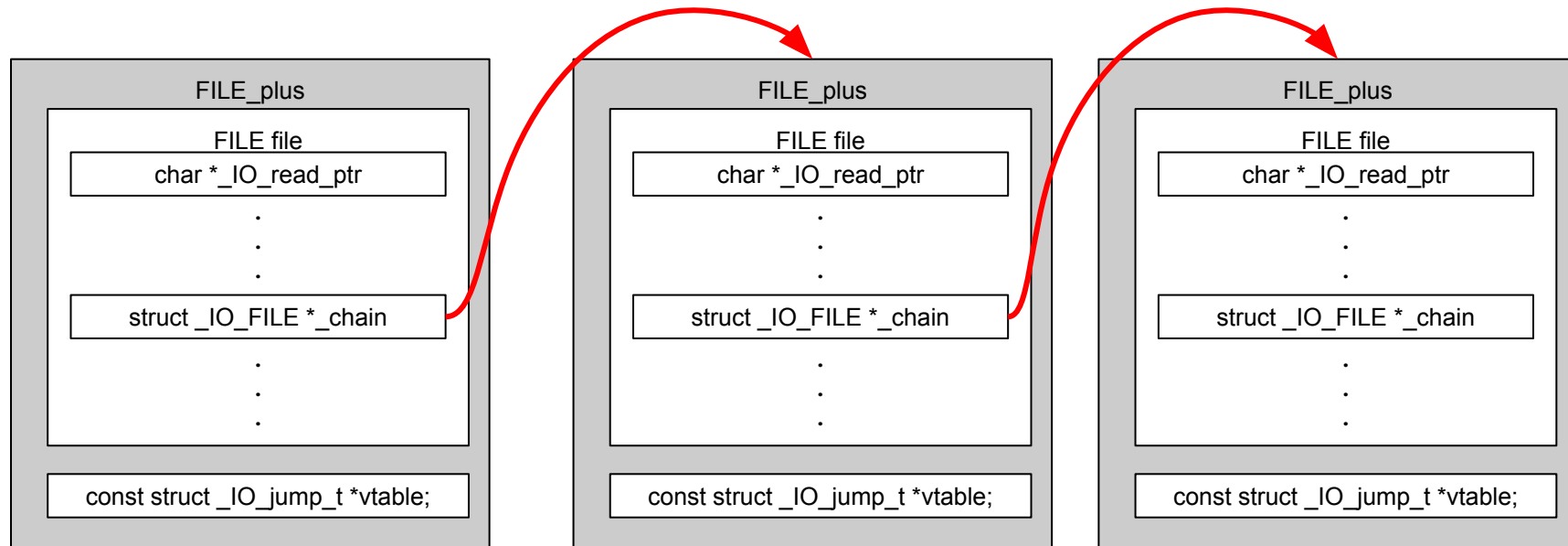
When space is limited, structs can be “overlapped” as long as the offsets accessed return reasonable values!



File struct - `_chain`

File structs are in a linked list via the `_chain` field!

This is used for cleanup/flushing at program exit



File Stream Oriented Programming

- Flushing streams relies on virtual functions
- Streams get flushed in order on `exit()`
- So cleverly overwriting FILE vtables in sequence
- Can call gadgets

Angry FSROP

FILE struct exploit techniques are still being discovered!

@kylebot in discord

<https://blog.kylebot.net/2022/10/22/angry-FSROP/>

